Evaluating the Collision Probability of Satellites in the Space Environment:

Abstract

This research aims to develop a robust and reliable model for predicting the probability of a satellite colliding with space debris. By leveraging advanced data analytics techniques and incorporating the latest advancements in space situational awareness, this study seeks to provide valuable insights into the factors influencing collision risk and enable proactive measures to safeguard critical space infrastructure.

Introduction:

Space debris, a growing problem, threatens the future of space exploration and the technology we rely on. Collisions between debris and active satellites can cause severe damage or mission failures, disrupting vital services like telecommunications, navigation, weather forecasting, and Earth observation. This debris consists of non-functional or useless objects in orbit, including damaged satellites, fragments from collisions and explosions. These objects pose a significant risk of collision, potentially leading to damage, mission loss, or catastrophic breakup. Such collisions can create more debris, triggering a chain reaction known as the Kessler syndrome, rendering certain orbital regions unusable for an extended period.

According to the European Space Agency (ESA), there are more than 34,000 objects larger than 10 cm, 900,000 objects between 1 and 10 cm, and 128 million objects smaller than 1 cm in orbit as of January 2020

. In figure shows the evolution of space debris and objects in orbit from 1960 to 2020. The x-axis represents the reference epoch (time period) in years, while the y-axis represents the count of objects.

The different colors represent different types of objects, including unidentified objects, payload mission-related objects, rocket mission-related objects, payload debris, rocket debris, rocket fragmentation debris, payload fragmentation debris, rocket body, and payload.

. This figure highlights the growing problem of space debris and the need for effective measures to mitigate its impact on future space activities.

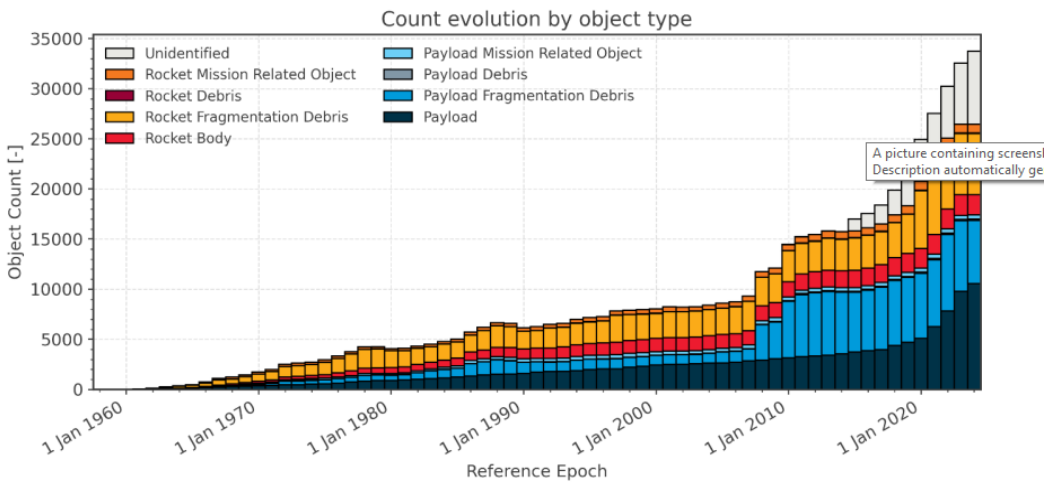


Figure 1 the evolution of the count of space debris and objects in orbit, categorized by type, from 1960 to 2020

The observations from the figure:

- Exponential growth: The total number of objects in orbit has increased
- exponentially over the years. This is due to increased space activities, such as satellite launches and missions, which have resulted in more debris being generated.
- Dominance of rocket-related debris: Rocket debris, including rocket bodies, fragmentation debris, and other components, make up a significant portion of the total debris. This is likely due to the frequent use of rockets for space launches and missions.
- Increasing number of unidentified objects: The category of unidentified objects has also grown over time. This may be due to the difficulty in tracking and identifying smaller pieces of debris.

- Impact of major events: The graph shows several peaks in the number of objects, which may be attributed to specific events, such as satellite collisions or intentional debris mitigation efforts.

To solve the problem of satellite collisions with space debris, there are two main approaches. The first is to reduce space debris through various techniques such as active debris removal, tether-based methods, and laser-based methods. The second approach is to use collision avoidance techniques, which will be the focus of this research.

Collision avoidance:

Collision avoidance generally requires the analysis and propagation of orbit information of resident space objects which is screened/compared against the propagated orbits of other objects, in order to identify close approaches.

collision avoidance procedures typically have four steps:

1. **Tracking and Monitoring**

- Space Situational Awareness (SSA): Utilize ground-based telescopes, radar systems, and satellite-based sensors to track the location and movement of debris in orbit.
- Data Fusion: Combine data from various sources to create a comprehensive picture of the debris environment.
- Collision Probability Analysis: Use algorithms to calculate the probability of a collision between a satellite and debris based on their orbits and relative velocities.

2. Risk Assessment

- Prioritization: Evaluate the potential consequences of a collision and prioritize objects that pose the highest risk to the satellite's mission.
- Mitigation Planning: Develop plans to address potential collision scenarios, including contingency measures and emergency procedures.

3. Maneuvering

- Orbital Adjustments: Make small adjustments to the satellite's orbit to avoid debris that is on a collision course.
- Emergency Maneuvers: In critical situations, perform larger maneuvers to rapidly change the satellite's orbit and avoid debris.
- Formation Flying: If multiple satellites are operating in close proximity, coordinate their movements to avoid debris collectively.

4. Autonomous Decision Making

- AI Integration: Equip satellites with artificial intelligence systems to enable autonomous decision-making in critical situations.
- Real-Time Analysis: Allow AI to process data from sensors and algorithms in real-time to identify potential threats and initiate appropriate maneuvers.

By following these procedures and staying updated on the latest advancements in space debris mitigation, spacecraft operators can significantly reduce the risk of collisions and ensure the long-term sustainability of space activities.[1]

Conjunction Data Message (CDM):

is a standardized format used in space operations to convey information about potential close encounters between orbiting objects, such as satellites and space debris. CDMs are generated when there is a predicted proximity between two objects in space, indicating that they may come within a specified threshold distance of each other, which could pose a collision risk.

This helps satellite operators assess the risk and decide whether to performcollision avoidance maneuver.

Conjunction Data Message include:

Identification: Details about the primary and secondary objects involved in the conjunction.

Time of Closest Approach (TCA): The predicted time when the two objects will be closest to each other.

Miss Distance: The estimated distance between the two objects at the TCA.

Uncertainty: Information about the uncertainties in the predicted positions and velocities of the objects.

Risk Assessment: Probability of collision and other relevant risk metrics.

After having the CDMs ,the main challenge is the analysis of the outcome of close encounters and the assessment of the information and predict the collision probability that future observations/CDMs will contain , do this by analyzing and learning the data patterns of previous events using AI methods and use that knowledge to predict the information in future observations of new events, which can give better insights into their outcome .

Data Analysis:

The dataset used is the one provided by ESA in 2019. It contains Conjunction Data Messages (CDMs) collected by the ESA Space Debris Office from 2015 to 2019, which support collision avoidance operations in the Low Earth Orbit (LEO) region. Each row in the dataset represents a single CDM, and there are 103 features in each CDM that provide detailed information about the close encounter. All the parameters in the CDMs are propagated to the time of closest approach (TCA). The CDMs are identified by an event ID, and messages from the same conjunction event are grouped under the same identifier. Therefore, each event forms a time series of CDMs, typically covering the week leading up to the TCA. [2]

The dataset includes many parameters, allowing for extensive statistical analysis.

In this section, only the most relevant information and analysis are presented to offer a clear understanding of the data and the problem context. Performing a correlation analysis of the dataset's features can also be useful, as it measures the statistical relationships between different parameters. This helps identify which features have a strong correlation with the target variable and highlights potential feature associations that should be further explored by AI models.

These two heatmaps visualize the correlation between various features in a dataset**.**

Heatmap 1

- **Color Scale:** The color scale ranges from dark green to dark red, indicating a strong positive correlation (dark green) to a strong negative correlation (dark red).
- **Features:** The x-axis and y-axis display the names of the features being analyzed.
- **Correlation Values:** The squares within the heatmap represent the correlation coefficients between the corresponding features. A value of 1 indicates a perfect positive correlation, while a value of -1 indicates a perfect negative correlation.

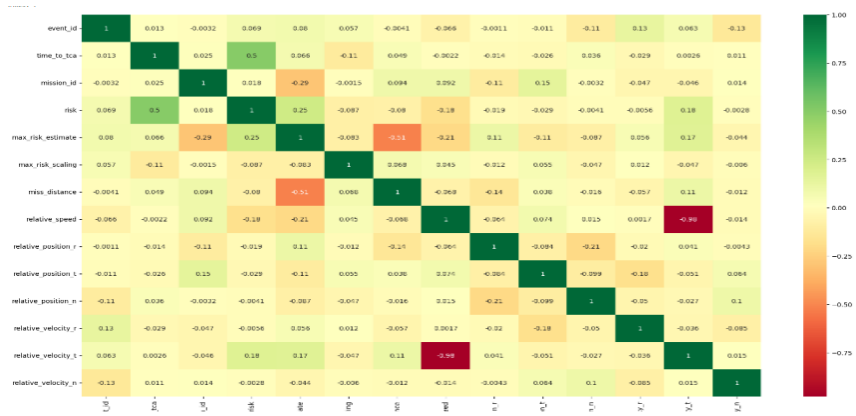


Figure 2heatmap to visualizes the correlation between various features in the dataset

Heatmap 2

- **Color Scale:** Like the first heatmap, the color scale ranges from dark green to dark red, representing strong positive to strong negative correlations.
- **Features:** The x-axis and y-axis show the names of the features, which appear to be different from those in the first heatmap.
- **Correlation Values:** The squares within the heatmap display the correlation coefficients between the corresponding features.

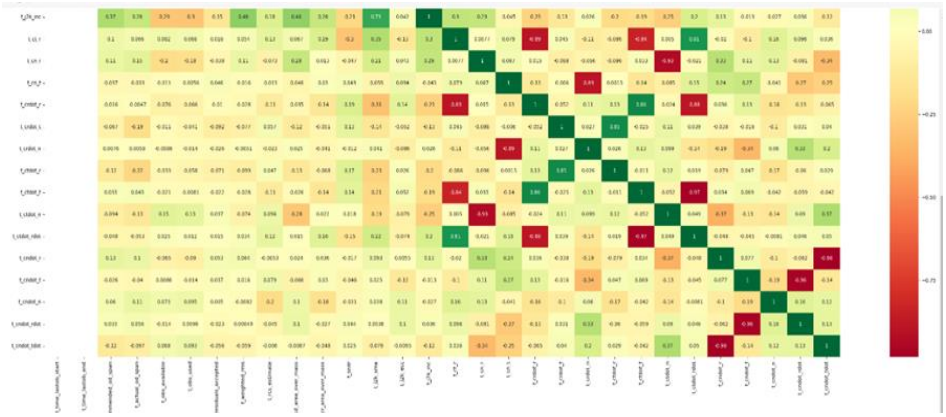


Figure 3heatmap to visualizes the correlation between various features in the dataset

The observations from the figures:

. Strong Positive Correlations: In both heatmaps, there are several squares that are dark green, indicating strong positive correlations between certain features. These features may be closely related or have a similar influence on the target variable.

. Strong Negative Correlations: Some squares in both heatmaps are dark red, suggesting strong negative correlations. This means that when one feature increases, the other tends to decrease.

. Weak Correlations: Many squares in both heatmaps are light green or yellow, indicating weak or no correlation between the corresponding features.

Overall, the heatmaps provide a visual representation of the relationships between the features in the dataset. This information can be valuable for understanding the underlying structure of the data and identifying potential dependencies or redundancies.

Correlation with risk:

These two bar charts visualize the correlation between the risk variable and other features in the dataset**.**

Chart 1

- X-axis: Displays the names of the features being analyzed.
- Y-axis: Represents the correlation coefficient between each feature and the target variable "risk."
- Bars: The height of each bar indicates the strength and direction of the correlation. A positive correlation is represented by a bar above the x-axis, while a negative correlation is represented by a bar below the x-axis.

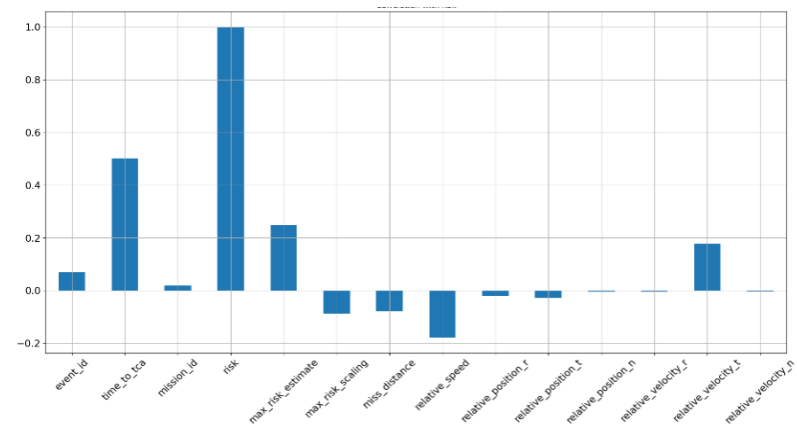


Figure 4correlation of features with risk

Chart 2

- X-axis: Displays the names of the features being analyzed (likely the same as in Chart 1).
- Y-axis: Represents the correlation coefficient between each feature and the target variable "risk."
- Bars: Like Chart 1, the height of each bar indicates the strength and direction of the correlation.

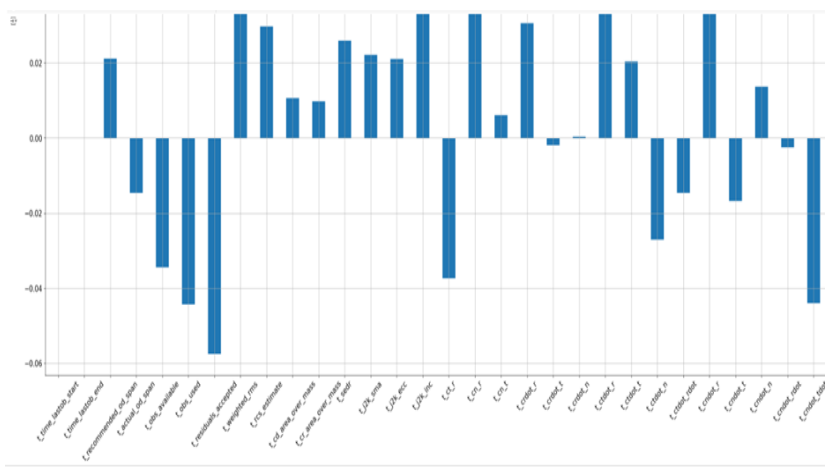


Figure 5correlation of features with risk

The observations from two bar charts:

. Strong Correlations: Some features have bars that extend significantly above or below the x-axis, indicating strong positive or negative correlations with the target variable "risk." These features may be particularly important in predicting or explaining the "risk."

. Weak Correlations: Other features have bars that are close to the x-axis, indicating weak or no correlation with the target variable. These features may be less relevant for predicting or explaining "risk."

. Positive vs. Negative Correlations:

The direction of the bars indicates whether the feature is positively or negatively correlated with "risk." A positive correlation means that as the feature increases, the "risk" also tends to increase. A negative correlation means that as the feature increases, the "risk" tends to decrease.

The charts provide a visual representation of the relationships between the features and the target variable "risk." This information can be valuable for understanding which features are most important for predicting or explaining the "risk," and for building models that can accurately predict the "risk" based on these features.

Correlation with miss distance:

The charts provide a visual representation of the relationships between the features and the target variable "miss distance." This information can be valuable for understanding which features are most important for predicting or explaining the "miss distance," and for building models that can accurately predict the "miss distance" based on these features.

Chart 1

- **X-axis:** Displays the names of the features being analyzed.
- **Y-axis:** Represents the correlation coefficient between each feature and the target variable "miss distance."

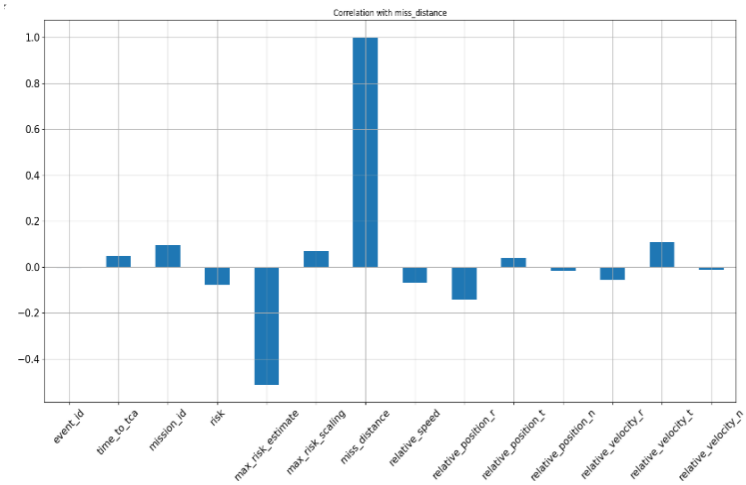


Figure 6Bar chart showing the correlation of various variables with 'miss_distance,' highlighting a perfect positive correlation for 'miss_distance' itself and a significant negative correlation for 'max_risk_scaling.'"

Bars: The height of each bar indicates the strength and direction of the correlation. A positive correlation is represented by a bar above the x-axis, while a negative correlation is represented by a bar below the x-axis.

Chart 2

- **X-axis:** Displays the names of the features being analyzed (likely the same as in Chart 1).
- **Y-axis:** Represents the correlation coefficient between each feature and the target variable "miss distance."
- **Bars:** Like Chart 1, the height of each bar indicates the strength and direction of the correlation.

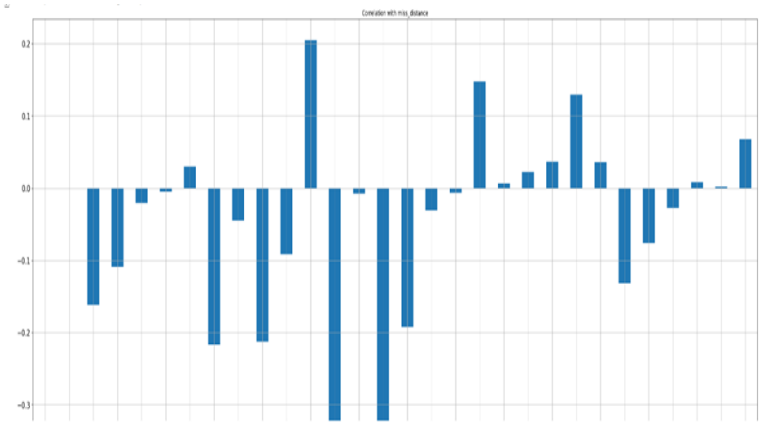


Figure 7Bar chart showing the correlation of various variables with 'miss_distance,' highlighting a perfect positive correlation for 'miss_distance' itself and a significant negative correlation for 'max_risk_scaling.'"

The observations from two bar charts:

. **Perfect Positive Correlation:** The feature "miss distance" itself has a bar that extends to the top of the chart, indicating a perfect positive correlation with itself. This is expected, as a variable is perfectly correlated with itself.

. **Significant Negative Correlation:** The feature "max_risk_scaling" has a bar that extends significantly below the x-axis, indicating a strong negative correlation with

"miss distance." This suggests that as "max_risk_scaling" increases, "miss distance" tends to decrease.

. Other Correlations: Other features also show varying degrees of positive or negative correlation with "miss distance." The heights of the bars indicate the strength of these correlations.

Correlation with mahalanobis_distance:

The charts provide a visual representation of the relationships between the features and Mahalanobis distance. This information can be valuable for understanding which features are most important in determining how far a data point is from the mean of the dataset.

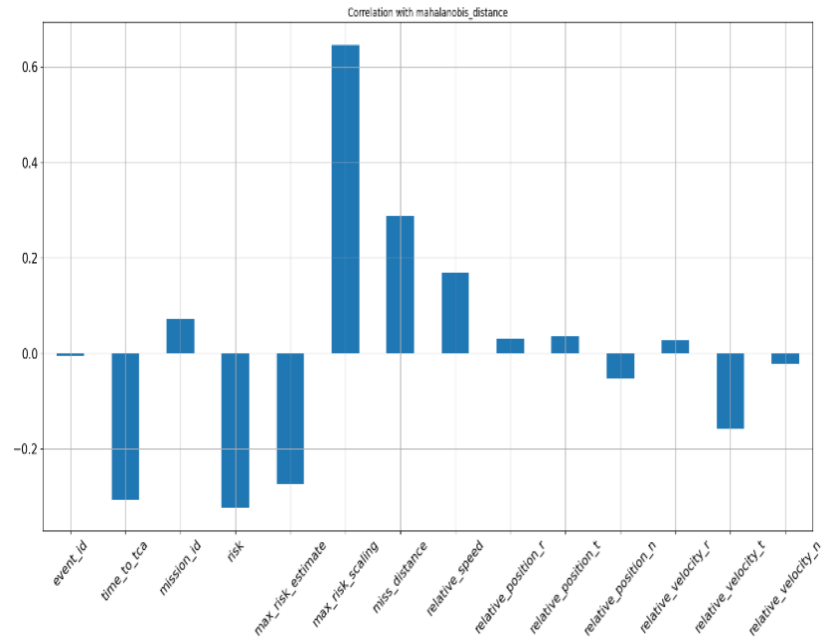


Figure 8The bar chart shows the correlation between various features in the dataset and Mahalanobis distance

Chart 1

- **X-axis:** Displays the names of the features being analyzed.
- **Y-axis:** Represents the correlation coefficient between each feature and Mahalanobis distance.
- **Bars:** The height of each bar indicates the strength and direction of the correlation. A positive correlation is represented by a bar above the x-axis, while a negative correlation is represented by a bar below the x-axis

Chart 2

- **X-axis:** Displays the names of the features being analyzed (likely the same as in Chart 1).
- **Y-axis:** Represents the correlation coefficient between each feature and Mahalanobis distance.

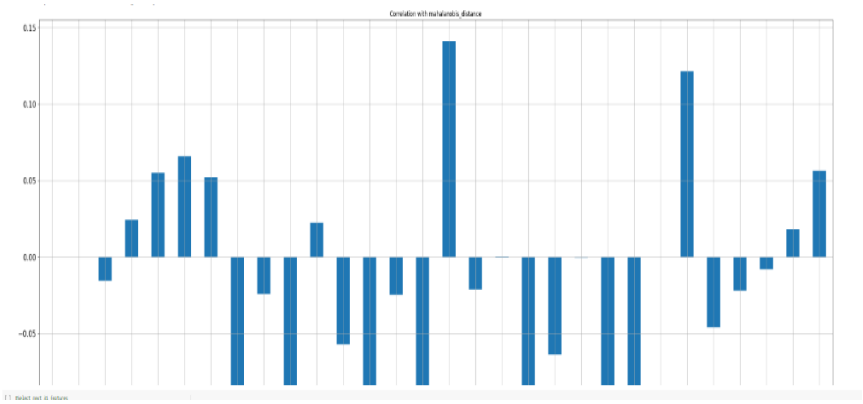


Figure 9The bar chart shows the correlation between various features in the dataset and Mahalanobis distance

- **Bars:** Like Chart 1, the height of each bar indicates the strength and direction of the correlation.

The observations from two bar charts:

. Correlations with Mahalanobis Distance: The charts show the correlation between each feature and Mahalanobis distance, which is a measure of how far a data point is from the mean of a dataset.

. Positive and Negative Correlations: Some features have positive correlations with Mahalanobis distance, indicating that they tend to be larger or smaller than the average data point. Other features have negative correlations, indicating that they tend to be closer to the average data point.

. V Correlations: The features with the tallest bars have the strongest correlations with Mahalanobis distance. These features may be particularly influential in determining how far a data point is from the mean.

Events and CDMs:

The bar chart visualizes the distribution of the number of CDMs (presumably, Clinical Decision Makers) received per event.

The Observations of bar chart:

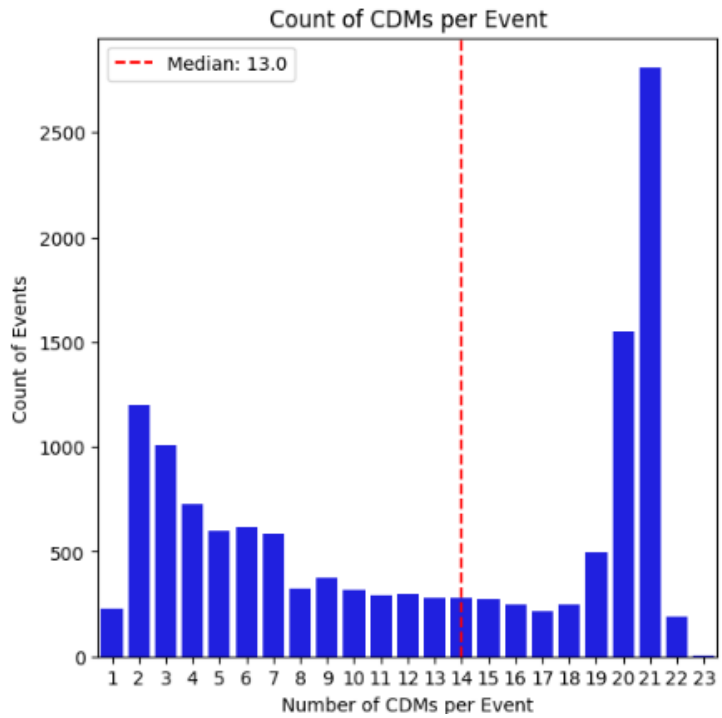


Figure 10Distribution of the number of CDMs received per event.

- **Average CDMs per Event:** The text above the chart indicates that there are 13,154 events and 162,634 CDMs in the dataset, giving an average of approximately 13 CDMs per event.
- **Distribution:** The bar chart shows the number of events that received a specific number of CDMs. For example, the tallest bar indicates that the most frequent number of CDMs received per event is around 23.
- **Median:** A vertical dashed line indicates the median number of CDMs received per event, which is 13. This means that half of the events received 13 or fewer CDMs, while the other half received 13 or more CDMs.
- **Skewness:** The distribution is skewed to the right, meaning that there are a few events that received a very large number of CDMs, while most events received a relatively small number of CDMs.

  The events should follow some constraints, since spacecraft operators need time to make a decision regarding the performance of an avoidance maneuver. These constraints are the following *:
  - the events must have at least 2 CDMs, one to learn and one to use as label.
  - the first DM must be released before the cut-off time (2 days until TCA)
  - the last CDM must be released within 1 day of the TCA.

TCA and Risk analysis:

The histogram provides a visual representation of the distribution of "last- risk" values in the dataset. This information can be valuable for understanding the overall risk level in the data and identifying any outliers or patterns.

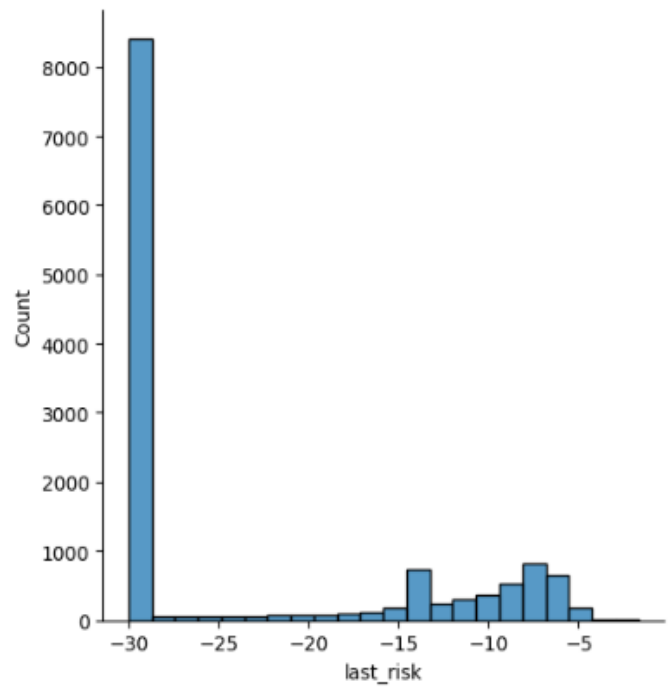


Figure 11 The histogram illustrates the distribution of the last_risk values, highlighting a significant concentration in the low-risk region, specifically at -30

The Observations of the histogram

- **Distribution:** The histogram shows the frequency of different values of "last_risk" within the dataset. The x-axis represents the range of "last_risk" values, while the y-axis represents the count of observations for each value.
- **Concentration in Low-Risk Region:** The chart indicates a significant concentration of "last_risk" values in the low-risk region, specifically at -30. This means that a large number of data points have a "last_risk" value close to -30.
- **Skewness:** The distribution is skewed to the right, meaning that there are a few data points with much higher "last_risk" values compared to the majority of the data.

miss distance and relative speed

The scatter plot provides a visual representation of the relationship between "relative_speed" and "miss_distance." This information can be valuable for understanding how these two variables are related and identifying any patterns or trends.

The Observations of the scatter plot:

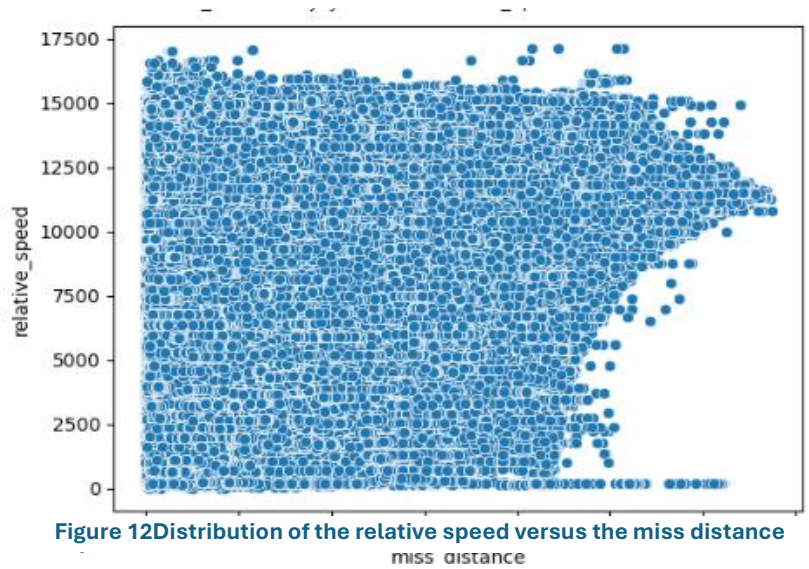


Figure 12Distribution of the relative speed versus the miss distance

. Scatter Plot: The plot displays each data point as a dot, where the x-coordinate represents the "miss_distance" value, and the y-coordinate represents the "relative_speed" value.

. Distribution: The data points appear to cluster in a specific pattern. There is a concentration of points towards the lower end of the "miss_distance" axis, with the "relative_speed" values varying across a wider range.

. Correlation: While it's difficult to determine a precise correlation from the scatter plot alone, there seems to be a general trend where higher "relative_speed" values are associated with lower "miss_distance" values. However, there is also a significant amount of scatter, indicating that the relationship is not perfectly linear.

The histogram illustrates the distribution of "relative_speed" among various events:

The histogram provides a visual representation of the distribution of "relative_speed" across different events. This information can be valuable for understanding the typical range of "relative_speed" values and identifying any outliers or patterns.

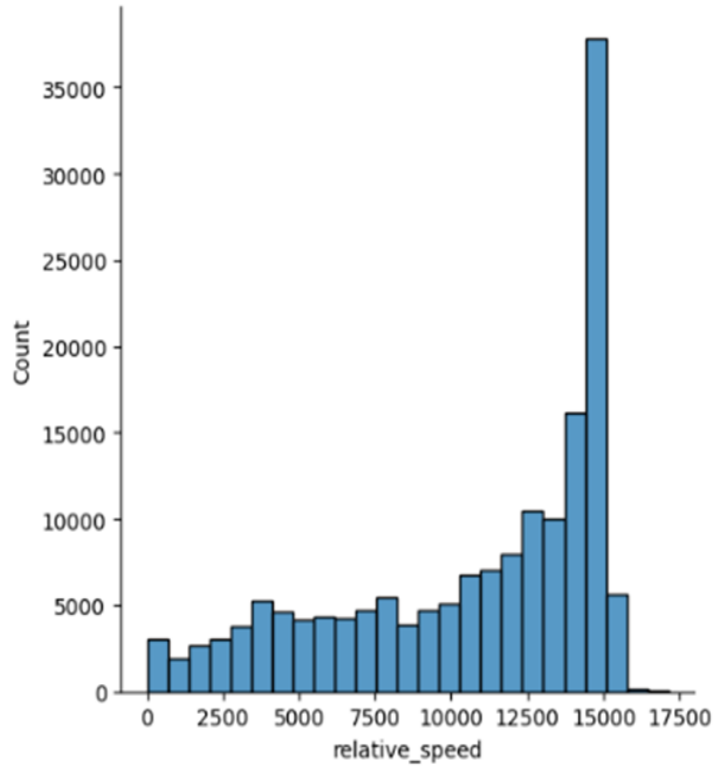


Figure 13the distribution of relative speed across different events

Observations

- **Distribution:** The histogram shows the frequency of different values of "relative_speed" within the dataset. The x-axis represents the range of "relative_speed" values, while the y-axis represents the count of observations for each value.
- **Concentration:** The chart indicates a significant concentration of "relative_speed" values around 15,000. This means that a large number of events have a "relative_speed" close to 15,000.
- **Skewness:** The distribution is skewed to the right, meaning that there are a few events with much higher "relative_speed" values compared to the majority of the data.

The plot illustrates the correlation between "relative_speed" and "azimuth angle."

The scatter plot provides a visual representation of the relationship between "relative_speed" and "azimuth angle." This information can be valuable for understanding how these two variables are related and identifying any patterns or trends.

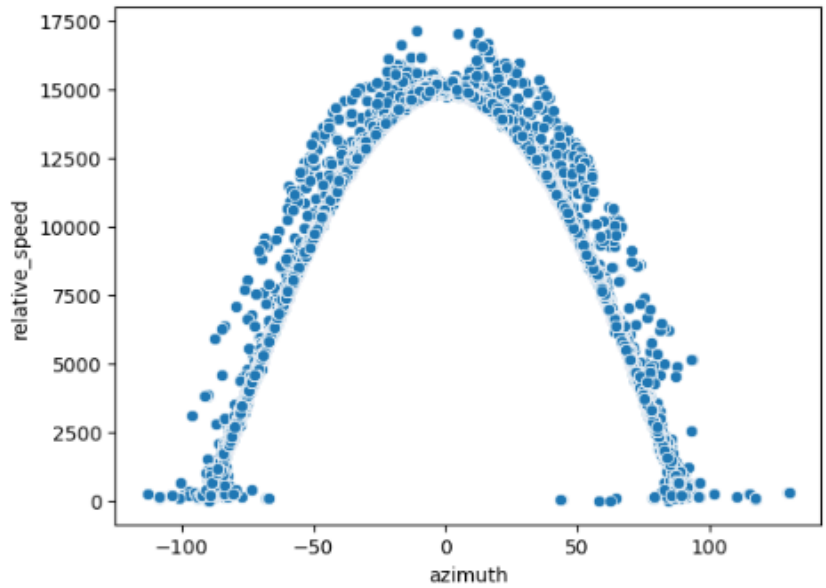


Figure 14distribution of the relative speed values against the azimuth angle

Observations

- **Scatter Plot:** The plot displays each data point as a dot, where the x-coordinate represents the "azimuth angle" value, and the y-coordinate represents the "relative_speed" value.
- **Distribution:** The data points appear to form a parabolic shape, with higher "relative_speed" values concentrated around the middle of the "azimuth angle" range and lower values towards the extremes.
- **Relationship:** The plot suggests a non-linear relationship between "relative_speed" and "azimuth angle." As the "azimuth angle" moves away from the center, the "relative_speed" tends to decrease, forming a parabolic curve.

Conjunction Location and Geometry

The histograms provide a visual representation of the distribution of these two variables in the dataset. This information can be valuable for understanding the typical range of values for orbital inclination and geocentric latitude, as well as identifying any patterns or trends.

Histogram 1: Distribution of Orbital Inclination

- **X-axis:** Represents the range of orbital inclination values.
- **Y-axis:** Represents the count of observations for each orbital inclination value.
- **Distribution:** The histogram shows a clear peak around 90 degrees, indicating that a large number of the data points have an orbital inclination close to 90 degrees. There are also smaller peaks around 88 and 92 degrees.

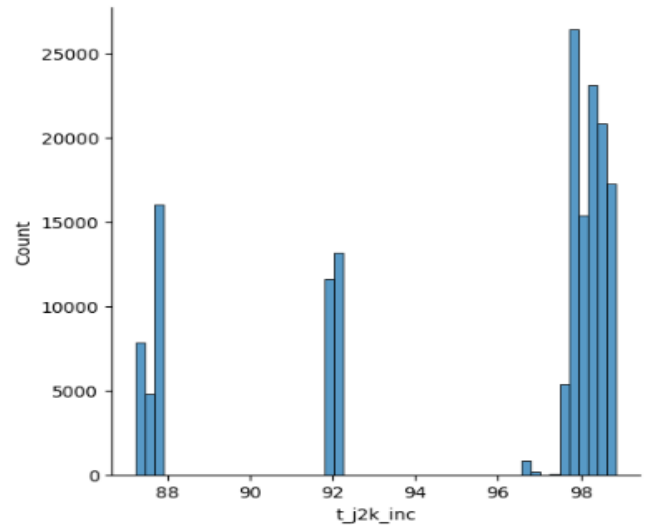


Figure 15Distribution of orbital inclination of the target

Histogram 2:

Distribution of Geocentric Latitude

- **X-axis:** Represents the range of geocentric latitude values.
- **Y-axis:** Represents the count of observations for each geocentric latitude value.

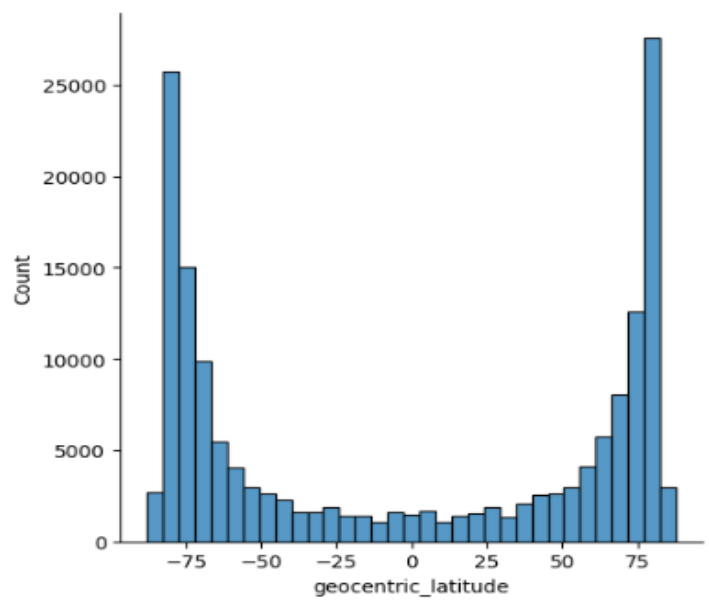


Figure 16Distribution of geocentric latitude

- **Distribution:** The histogram shows a clear peak around 0 degrees, indicating that a large number of the data points have a geocentric latitude close to 0 degrees. The distribution is generally symmetric around 0 degrees, with a smaller peak around -50 degrees and a wider spread of values towards the higher latitudes.

Object properties:

This bar chart represents the distribution of chaser object types. The x-axis lists the different types of chaser objects, while the y-axis represents the count of each type.

Here's a breakdown of the distribution:

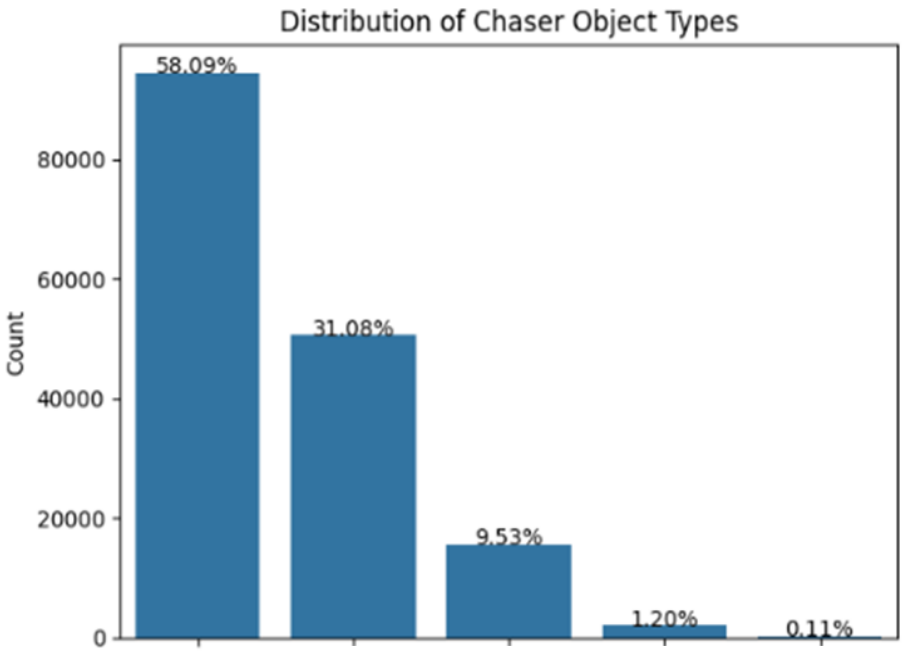


Figure 17 represents the distribution of chaser object types

- **DEBRIS:** This is the most common type of chaser object, making up 58.09% of the total.
- **UNKNOWN:** This category accounts for 31.08% of chaser objects, indicating that the type of many objects is unidentified.
- **PAYLOAD:** 9.53% of chaser objects are classified as payloads.
- **ROCKET BODY:** This category comprises 1.20% of the total.
- **TBA:** The smallest category, TBA (To Be Announced), represents 0.11% of chaser objects.

Overall, the chart shows that the majority of chaser objects are debris, followed by unknown objects. The remaining categories, payloads, rocket bodies, and TBA, make up a smaller proportion of the total.

Now that we have talked about the analysis of the data let us see how can use Python to train the modules to predict the risk. We will try two modules to compare the accuracy between them. We will start explaining the code**.**

Let us start with the main libraries that are used for the modules:

Pandas:

Pandas is a Python library for working with data sets. It allows us to analyze big data and make conclusions based on statistical theories. Since we need to work with a CSV file, we need to use pandas to examine, clean, explore, and manipulate data. Pandas can clean messy data sets and make them readable and relevant.

"Pandas" is referenced to both "Panel Data", and "Python Data".[3]

matplotlib:

Matplotlib is a library that was created by John D. Hunter and is written in Python, C, Objective-C, and JavaScript. It is a low-level graph plotting library that is used as a visualization utility. It can be used freely since it is an open source. [4]
We name the matplotlib in the code as plt and pyplot submodule has most of the Matplotlib utilities and has a collection of functions that make matplotlib work like MATLAB. We will use it since we need it to plot graphs of the data.[5]

NumPy:

NumPy is a library that was created in 2005 by Travis Oliphant and is also an open-source library. It is used for working with arrays since it provides a multidimensional array object. It also provides various derived objects (such as masked arrays and matrices). It can do fast operations on arrays (faster than the list), this includes mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation, and much more.[6]

seaborn:

Seaborn is a Python library that is used for data visualization based on Matplotlib. It is better than matplotlib because it provides a much easier high-level interface for drawing attractive and informative statistical graphics such as heatmap () is used for plotting correlation and confusion matrix. [7]

Scikit-learn:

Scikit-learn, or sklearn was first released in 2010, is also an open-source library for Python that It implements numerous data modeling and machine learning algorithms which help us provide many features such as classification, regression, clustering, vector machines, random forests, gradient boosting, k-means. It can work with other Python libraries such as matplotlib for plotting, NumPy for arrays and vectors, Pandas for dataframes, and many more.

We will use sklearn for random forest regression, and for Model Selection to Compare, validate, and choose parameters and Model Selection Comparing, validating, and choosing parameters and models. [8]

How to measure error:

There are many ways to measure the error. Depending on the type of the module we use different types of error measurement to check the performance. We will use regression modules so we will need errors to measure the performance of these types of modules.

Here are the types:

. Mean squared error

. Root Mean Squared Error (RMSE):

. mean absolute error

Mean squared error:

Mean squared error is a technique that helps us measure the error of how close a regression line is to a set of data points, as shown in Figure 1. It is a risk function corresponding to the expected value of the squared error loss

By squaring the errors, it penalizes even if they are small. This leads to an overestimation of how bad the model is. However, it's fundamentally more prone to outliers than other metrics

we can calculate it from the equation:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(\hat{Y}_i - Y_i)^2$$

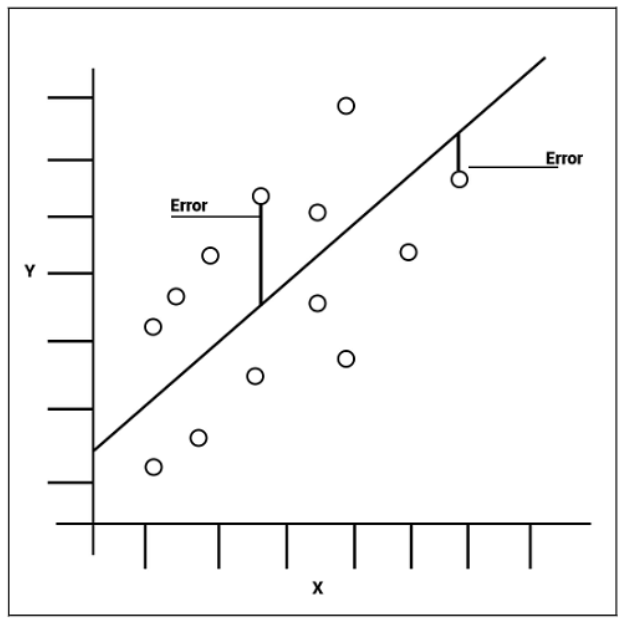
Where:

n: number of data

i: index number

Ŷ: predicted value

Y: real value from the training data



(Figure 18)Scatterplot with a regression line and error terms.

Root Mean Squared Error (RMSE):

The square root of the MSE. It is more interpretable and less sensitive to outliers, which is why it is often preferred over MSE. And it has the differentiable property of MSE. Since the scale is now the same as the random variable, error interpretation can be done smoothly, and it is less prone to struggle in the case of outliers since scale factors are normalized.

we can calculate it from the equation:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{Y}_i - Y_i)^2}$$

Where:

n: number of data

i: index number

Ŷ: predicted value

Y: real value from the training data

mean absolute error:

Mean squared error is a technique for measuring the error of how close a regression line is to a set of data points, as shown in(**Figure 19**). It is a risk function corresponding to the expected value of the absolute value of error loss.

It does not exaggerate errors since it is more robust towards outliers than MSE. MAE provides a measure of how far the predictions are from the actual output, but it does not specify the direction of the error (under-prediction or over-prediction). MAE is non-differentiable as opposed to MSE, which is differentiable.

we can calculate it from the equation:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|\hat{Y}_i - Y_i|$$

Where:

n: number of data

i: index number

Ŷ: predicted value

Y: real value from the training data

R2 score

The R2 score, also called the coefficient of determination, is a statistical measure used to evaluate the performance of a linear regression model. It is the amount of variation in the output-dependent attribute that is predictable from the input independent variable(s). Depending on the ratio of total deviation of results described by the model, it is used to check how well-observed the model reproduces results.
It helps us measure "How much of the total variation in Y(target) is explained by the variation in X (regression line)"
it could mislead us into believing that the model is improving when the score is increasing but in reality, the learning is not happening. This can happen in the case of model overfitting the data,[9]

Mathematical Formula:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Where:

SS_{res}: is the sum of squares of the residual errors.

SS_{tot}: is the total sum of the errors.[10]

The main code:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

```
train_data= pd.read_csv('train_data.csv')
train_data
```

	event_id	time_to_tca	mission_id	risk	max_risk_estimate	max_risk_scaling	miss_distance	relative_speed	relative_position_r	relative_position_t
0	0	1.566798	5	-10.204955	-7.834756	8.602101	14923.0	13792.0	453.8	5976.6
1	0	1.207494	5	-10.355758	-7.848937	8.956374	14544.0	13792.0	474.3	5821.2
2	0	0.952193	5	-10.345631	-7.847406	8.932195	14475.0	13792.0	474.6	5796.2
3	0	0.579669	5	-10.337809	-7.845880	8.913444	14579.0	13792.0	472.7	5838.9
4	0	0.257806	5	-10.391260	-7.852942	9.036838	14510.0	13792.0	478.7	5811.1
...	...	...	...	...	...	...	...	...	...	...
162629	13153	3.029751	1	-7.108630	-5.142668	7.561579	2307.0	14976.0	-472.1	-159.9
162630	13153	2.799253	1	-7.070070	-5.137869	7.471857	2254.0	14976.0	-469.7	-163.3
162631	13153	2.385399	1	-7.066209	-5.137034	7.463470	2259.0	14976.0	-469.3	-158.8
162632	13153	2.043339	1	-7.028307	-5.131297	7.378397	2257.0	14976.0	-465.2	-160.3
162633	13153	1.618095	1	-7.172372	-5.152181	7.705750	2185.0	14976.0	-479.6	-156.0

162634 rows × 103 columns

we call the main libraries that we will need their functions.we use Pandas library to read the data from the file ‘train_data.csv’ and save it in a data frame variable called train_data. When we print this data it appears as a table of data, showing the first 5 columns of the table (head) and the last 5 columns of that table (tail).

```
train_data.isna().sum()
```

```
event_id                0
time_to_tca             0
mission_id              0
risk                    0
max_risk_estimate       0
                      ...
c_sigma_ndot         9241
F10                  6822
F3M                  6822
SSN                  6822
AP                   6822
Length: 103, dtype: int64
```

Pandas library function that helps us calculate the total number of missing values (NaNs) in each column of a DataFrame of variable train_data.

train_data = train_data.interpolate()

it is a Pandas function that interpolates missing values in a DataFrame of variable train_data.

```
train_data = train_data.interpolate()
train_data.isnull().sum()
```

```
event_id             0
time_to_tca          0
mission_id           0
risk                 0
max_risk_estimate    0
                    ..
c_sigma_ndot         0
F10                  0
F3M                  0
SSN                  0
AP                   0
Length: 103, dtype: int64
```

```
df_copy = train_data[['event_id','time_to_tca','risk','max_risk_estimate', 'max_risk_scaling', 'miss_distance',
                      'relative_speed', 'relative_position_r', 'relative_position_t','relative_position_n',
                      'relative_velocity_r', 'relative_velocity_t','relative_velocity_n',
                      't_time_lastob_start', 't_time_lastob_end', 't_recommended_od_span','t_actual_od_span',
                      't_obs_available','t_obs_used', 't_residuals_accepted', 't_weighted_rms','t_rcs_estimate',
                      't_cd_area_over_mass', 't_cr_area_over_mass','t_sedr', 't_j2k_sma', 't_j2k_ecc', 't_j2k_inc',
                      't_ct_r', 't_cn_r', 't_cn_t', 't_crdot_r', 't_crdot_t', 't_crdot_n', 't_ctdot_r','t_ctdot_t',
                      't_ctdot_n', 't_ctdot_rdot', 't_cndot_r','t_cndot_t', 't_cndot_n', 't_cndot_rdot',
                      't_cndot_tdot', 'c_time_lastob_start', 'c_time_lastob_end','c_recommended_od_span',
                      'c_actual_od_span','c_obs_available', 'c_obs_used', 'c_residuals_accepted','c_weighted_rms',
                      'c_cd_area_over_mass','c_cr_area_over_mass', 'c_sedr', 'c_j2k_sma', 'c_j2k_ecc', 'c_j2k_inc',
                      'c_ct_r', 'c_cn_r', 'c_cn_t','c_crdot_r', 'c_crdot_t', 'c_crdot_n', 'c_ctdot_r', 'c_ctdot_t',
                      'c_ctdot_n','c_ctdot_rdot', 'c_cndot_r','c_cndot_t', 'c_cndot_n','c_ctdot_rdot', 'c_cndot_r',
                      'c_cndot_t', 'c_cndot_n','c_cndot_rdot', 'c_cndot_tdot', 't_span', 'c_span', 't_h_apo',
                      't_h_per', 'c_h_apo', 'c_h_per', 'geocentric_latitude', 'azimuth', 'mahalanobis_distance',
                      't_sigma_r', 'c_sigma_r', 't_sigma_t', 'c_sigma_t', 't_sigma_n','c_sigma_n', 't_sigma_rdot',
                      'c_sigma_rdot', 't_sigma_tdot', 'c_sigma_tdot', 't_sigma_ndot',
                      'c_sigma_ndot', 'F10', 'F3M', 'SSN', 'AP']]
df_copy
```

a Pandas function calculates the total number of missing values (NaNs) in each column of the Data Frame train_data after interpolating the missing values.

It creates a data frame variable called df_copy to store all the features we need for the training.

from sklearn.model_selection import train_test_split

it calls the train_test_split functions from the sklearn.model_selection module. They are used for splitting a dataset into training and testing sets. That is why these functions are crucial tools in machine learning.

X = df_copy.drop(['risk'], axis=1)

It removes (drops) the 'risk' column from the DataFrame variable df_copy and saves the result to a new variable dataframe X. The (axis) argument specifies whether it is a column or row that should be removed.

axis=0 for removing a raw
axis=1 for removing a column

y = df_copy.risk

it creates a new variable that stores the extraction of the 'risk' column from the DataFrame df_copy

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=100)

The train_test_split function from the Scikit-learn library splits a dataset randomly into training and testing sets

- **X:** The variable Data Frame contains the features (independent variables) of the dataset.
- **y:** This is the array containing the target variable of the dataset in our case the ‘risk’.
- test_size=0.4**:** This argument specifies the portion of the data that will be used for the testing set. In this case, 40% of the data will be used for testing.
- random_state=100**:** we want to ensure that the split is reproducible, so we use this argument to specify the random seed for the split,

which. means that we will get the same training, and test sets every time we run the code with the same random state.

In the module by using random forest regression:

1. **Code:**

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error,mean_absolute_error,r2_score

 # create regressor object
regressor = RandomForestRegressor(n_estimators = 100, random_state = 0)
regressor.fit(X_train,y_train)
predR = regressor.predict(X_test)
rmseR = np.sqrt(mean_squared_error(y_test, predR))
mseR=mean_squared_error(y_test,predR)
maeR=mean_absolute_error(y_test,predR)
r2R=r2_score(y_test,predR)
result1=[rmseR,mseR,maeR,r2R]
df_R = pd.DataFrame({'RandomForestRegressor':result1},index=['RMSE:','MSE:','MAE:','R2-score:'])
df_R
```

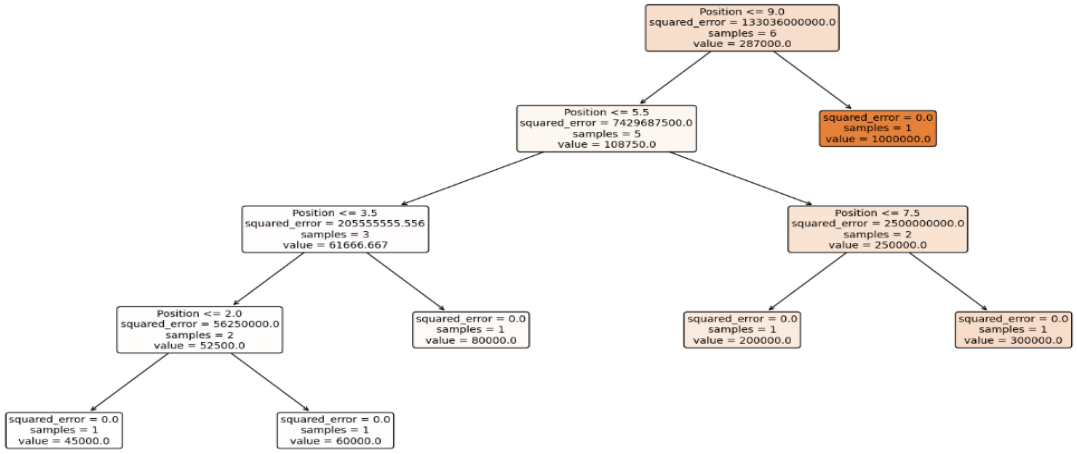
explain:

This library is a multipurpose machine-learning technique for predicting numerical values. It uses multiple decision trees and combines its predictions to reduce overfitting and improve accuracy. It is an easy and optimized approach.

Ensemble Learning

It is a machine-learning technique that uses predictions from multiple models to create more accurate and stable predictions.

Ensemble learning leverages the collective intelligence of various models rather than relying on a single model to improve the overall performance of the learning system.



(Figure 20)Decision tree from random forest

The code:

regressor = RandomForestRegressor(n_estimators= 100, random_state = 0) creates a class of random forest regression model with 100 decision trees and a fixed random seed for reproducibility. This model can then be trained on a dataset and used to make predictions on new data.

Here is a breakdown of the parameters:

- n_estimators=100: This parameter specifies the number of decision trees that will be created in the random forest. A higher number of trees generally leads to better performance but training time increases.
- random_state=0: When building trees, it controls the randomness of the bootstrapping of the samples used.[11]

code:

regressor.fit(X_train,y_train)

regressor: this is the object we created from the class RandomForestRegressor.

fit (): This is a method associated with the object that performs the model training. It trains the random forest regression model on the training data. It trains the input data features (X_train) with the value that should be predicted or the target value and it is a one-dimensional array (y_train). By joining predictions from multiple trees, the Random Forest Regressor aims to achieve strong and accurate predictions.

When running this code, the object stores the learned model parameters which the trained decision tree, but it does not return or print anything.

Code:

predR = regressor.predict(X_test)

Predict regression target for **X_test** then the predicted regression target value is then saved in a variable called **predR**. In our case is one vector of the predicted values of the risk. The predicted values of an input sample are computed as the mean predicted regression targets of the trees in the forest.

Code: mseR=mean_squared_error(y_test,predR)

This code is used to calculate the error by the mean squared error between the predicted value (predR) and the real value (y_test) in the random forest regression problem and save the result in a variable named **mseR**

Code: rmseR = np.sqrt(mean_squared_error(y_test, predR))

This code is used to calculate the error by the square root of the mean squared error between the predicted value (predR) and the real value (y_test) in the random forest regression problem and save the result in a variable named **rmseR.**

Code: maeR=mean_absolute_error(y_test,predR)

This code is used to calculate the error by the mean absolute error between the predicted value (predR) and the real value (y_test) in the random forest regression problem and save the result in a variable named **maeR.**

Code: r2R=r2_score(y_test,predR)

This code is used to calculate the error by the coefficient of determination between the predicted value (predR) and the real value (y_test) in the random forest regression problem and save the result in a variable named **r2R.**

Code: result1= [rmseR, mseR, maeR, r2R]

This code stores the variables of evaluation metrics for the regression model the mean squared error, square root of the mean squared error, mean absolute error, and the coefficient of determination in list variable **result1**.

Code: df_R = pd.DataFrame ({'RandomForestRegressor':result1}, index=['RMSE:','MSE:','MAE:','R2-score:'])

This code creates a pandas DataFrame variable **df_R** to store the evaluation metrics for the Random Forest Regressor model.[12]

second module:

```
from sklearn.svm import SVR
regressor = SVR(kernel ='sigmoid')

regressor.fit(X_train,y_train)
y_pred = regressor.predict(X_test)
rmseSV = np.sqrt(mean_squared_error(y_test, y_pred ))
mseSV=mean_squared_error(y_test,y_pred )
maeSV=mean_absolute_error(y_test,y_pred )
r2SV=r2_score(y_test,y_pred)
result2=[rmseSV,mseSV,maeSV,r2SV]
df_SV = pd.DataFrame({'RandomForestRegressor':result2},index=['RMSE:','MSE:','MAE:','R2-score:'])
df_SV
```

explain & used library:

Regression challenges are handled by support vector regression (SVR), a support vector machine (SVM). For a given input value, it looks for a function that best predicts the continuous output value.

In SVR, both linear and non-linear kernels can be used. A non-linear kernel is a more complicated function that can capture more complex patterns in the data, whereas a linear kernel is only a simple dot product between two input vectors. The complexity of the task and the properties of the data determine the kernel to use.

The "SVR" class in the scikit-learn Python package allows you to do SVR with either a linear or non-linear "kernel." You can select "linear" or "RBF" (radial basis function) as the kernel option to specify the kernel.

Support Vector Regression (SVR) is a supervised machine learning algorithm employed for regression tasks. Unlike traditional regression methods, SVR seeks a function that optimally predicts continuous output values based on given input data. This function is determined by finding a hyperplane in a high-dimensional space that maximizes the margin between data points.

SVR offers flexibility through its ability to use both linear and nonlinear kernels. A linear kernel simply computes the dot product of input vectors, while a nonlinear kernel, such as the Radial Basis Function (RBF), can capture more complex patterns. The choice of kernel is influenced by the data's characteristics and the task's complexity.

In Python's scikit-learn library, SVR is implemented using the SVR class. The kernel parameter can be set to 'linear' or 'rbf' to specify the kernel type.

Key Concepts:

Support Vectors: These are data points that lie closest to the hyperplane and significantly influence the model's decision boundary.

Hyperplane: A decision boundary in a high-dimensional space that separates data points based on their output values.

Margin: The distance between the hyperplane and the nearest data points. SVR aims to maximize this margin.

Regularization: SVR uses regularization to prevent overfitting by controlling the model's complexity. The C parameter in scikit-learn's SVR class balances the trade-off between fitting the training data and avoiding overfitting.[13]

the code:

```
from sklearn.svm import SVR
regressor = SVR (kernel ='sigmoid')
```

The first thing we will do is call the the library for the support vector machine from the sklearn.svm so we can use its module.

Then the second code we create a Support Vector Regression (SVR) model which is a supervised machine learning algorithm used for regression tasks. And save it in a variable called regressor

kernel='sigmoid': This parameter specifies in the SVM to use the kernel function. In this case, a sigmoid kernel is chosen. The sigmoid kernel is a non-linear kernel function that maps input data into a higher-dimensional feature space, allowing for more complex decision boundaries we often use it if we predict the relationship between features and the target variable may be non-linear and may be approximated by a sigmoid function.

regressor.fit(X_train,y_train)

regressor: this is the object we created from the support vector regression (SVR).

fit (): This is a method associated with the object that performs the model training. It trains the SVM model on the training data. It trains the input data features (X_train) with the value that should be predicted or the target value and it is a one-dimensional array (y_train).

by using the sigmoid kernel, It learns to plot the training data points into a higher-dimensional feature space and tries to find a hyperplane (decision boundary) in this space that separates the data points according to their target values while minimizing the error for the training samples.
The model internalizes the knowledge gained from the training data by adjusting various parameters, like the weights associated with different features and the kernel parameters.

Code:

y_pred = regressor.predict(X_test)

Predict the regression target for **X_test** then the predicted regression target value is then saved in a variable called **y_pred**. In our case is one vector of the predicted values of the risk. The predicted values of an input sample are computed as the mean predicted regression targets of the trees in the forest.

Code: mseSV=mean_squared_error(y_test,y_pred)

This code is used to calculate the error by the mean squared error between the predicted value (predSV) and the real value (y_test) in the random forest regression problem and save the result in a variable named **mseSV**

Code: rmseSV = np.sqrt(mean_squared_error(y_test, predSV))

This code is used to calculate the error by the square root of the mean squared error between the predicted value (y_pred) and the real value (y_test) in the random forest regression problem and save the result in a variable named **rmseSV.**

Code: maeSV=mean_absolute_error(y_test, y_pred)

This code is used to calculate the error by the mean absolute error between the predicted value (y_pred) and the real value (y_test) in the random forest regression problem and save the result in a variable named **maeSV.**

Code: r2R=r2_score(y_test,y_pred)

This code is used to calculate the error by the coefficient of determination between the predicted value (y_pred) and the real value (y_test) in the random forest regression problem and save the result in a variable named **r2SV.**

Code: result1= [rmseSV, mseSV, maeSV, r2SV]

This code stores the variables of evaluation metrics for the regression model the mean squared error, square root of the mean squared error, mean absolute error, and the coefficient of determination in list variable **result2**.

Code: df_SV = pd.DataFrame ({'RandomForestRegressor':result2}, index=['RMSE:','MSE:','MAE:','R2-score:'])

This code creates a pandas DataFrame variable **df_SV** to store the evaluation metrics for the Random Forest Regressor model.

Comparison in accuracy:

confusion matrix of random forest regeneration

	RandomForestRegressor
RMSE:	0.103305
MSE:	0.010672
MAE:	0.025896
R2-score:	0.999894

This matrix shows that this module achieved high accuracy since the error is low in RMSE, MSE, MAE, and R2 is high

Confusion matrix of SVM

RMSE:	18.595200
MSE:	345.781453
MAE:	11.138926
R2-score:	-2.433710

This matrix shows that this module achieved low accuracy since the error is high in RMSE, MSE, MAE, and R2 is low.

This matrix shows that this module achieved low accuracy since the error is high in RMSE, MSE, MAE, and R2 is low.
this might be because the sigmoid kernel might not suit this data's underlying relationships, and the data fit more for the linear relationships.

References

[1]: https://www.esa.int/Space_Safety/Space_Debris/Reentry_and_collisance

[2]: https://kelvins.esa.int/collision-avoidance-challenge/data/

[3]: https://www.w3schools.com/python/pandas/pandas_intro.asp

[4]: https://www.w3schools.com/python/matplotlib_intro.asp

[5]: https://matplotlib.org/3.7.1/tutorials/introductory/pyplot.html

[6]: https://numpy.org/doc/stable/

[7]: https://seaborn.pydata.org/

[8]: https://domino.ai/data-science-dictionary/sklearn

[9]: https://neptune.ai/blog/performance-metrics-in-machinecomplete-guide

[10]: https://www.geeksforgeeks.org/python-coefficient-of-determinatiscore/

[11]: https://scikitlearn.org/1.5/modules/generated/sklearn.ensemble.r.html

[12]:https://scikitlearn.org/1.5/modules/generated/sklearn.ensemble.RandomForestRegressr.html

[13]: https://www.geeksforgeeks.org/support-vector-regression-svr-using-linear-and-non-linear-kernels-in-scikit-learn/

Figure 21): https://www.simplilearn.com/tutorials/statistics-tutorial/mean-squared-error

(Figure 22): https://www.geeksforgeeks.org/random-forest-regression-in-python/